



AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH (AIUB)

Department of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Summer 2025-2026

Section: *E*

Group No: *03*

Project Report On

Smart Daily Task & Habit Management System

Supervised By

Arjun Kumar Bose Arnob

Submitted By:

Name	ID
Shuvo Chandra Mali	21-45971-3
Afif Karim Ryang	23-51249-1
As Sadik Oshan Ahmed	24-58086-2
Siddhartha Chandra Paul	24-60418-3

Obtained Marks for CO2 and CO4 (Description given in the following page)					
Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria (CO2)	Total =		Evaluation Criteria (CO4)		Total =
Requirement fulfillment			Design & Innovation		
Validation			Teamwork & Ethics		
Verification			Communication		

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Requirement fulfillment	Fails to demonstrate any understanding of real-life scenario-based project development or functional requirement identification. There is no attempt to depict a project or identify functional requirements accurately.	Demonstrates limited understanding of real-life scenario-based project development and functional requirement identification. The project depicted lacks coherence or relevance to real-life scenarios, and functional requirements are inaccurately identified or insufficiently described.	Presents a basic depiction of a real-life scenario-based project and identifies some functional requirements. However, the project lacks depth or complexity, and some functional requirements may be vaguely defined or missing key details.	Effectively demonstrates a realistic scenario-based project and accurately identifies most functional requirements. The project is well-developed with appropriate complexity, and functional requirements are clearly articulated with relevant details.	Exhibits an exceptional understanding of real-life scenario-based project development and accurately identifies all functional requirements. The project is meticulously developed with thorough attention to detail, reflecting a comprehensive understanding of Object-Oriented Programming project development activities.
Validation	Fails to demonstrate any understanding or implementation of validation forms in their system. There is no attempt to deal with data validation, and validation requirements are completely ignored or incorrectly applied.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Effectively demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall

					reliability and integrity of the system.
Verification	Fails to demonstrate any attempt to verify the system data or functional requirements. There is no evidence of understanding or implementation of verification processes, and data flow is not considered.	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts are incomplete or inaccurate, and there is insufficient consideration given to ensuring data integrity and functionality.	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may be inconsistent or lack thoroughness, and there may be gaps in ensuring proper functional requirements and data flow.	Identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and thorough, with attention to ensuring data integrity and appropriate data flow within the system.	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

CO4: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	Evaluation Definition				
Requirement fulfillment	Fails to demonstrate any attempt to perform or solve system functional requirements. There is no evidence of addressing real-life scenario-based projects or identifying functional requirements for Object-Oriented Programming project development activities.	Demonstrates limited understanding of system functional requirements and real-life scenario-based projects. Attempts to perform or solve functional requirements may be incomplete or inaccurately applied, and there is insufficient consideration given to proper identification of requirements.	Shows a basic understanding of system functional requirements and attempts to perform or solve them. However, the execution may lack depth or thoroughness, and there may be gaps in addressing all aspects of real-life scenario-based projects or properly identifying functional requirements.	Effectively demonstrates the performance and solution of system functional requirements for real-life scenario-based projects. Requirements are identified with clarity and relevance, and the execution is mostly accurate and comprehensive, reflecting a solid understanding of Object-Oriented Programming project development activities.	Exhibits an exceptional understanding of system functional requirements and adeptly performs and solves them for real-life scenario-based projects. Requirements are meticulously identified and executed with thorough attention to detail, showcasing a comprehensive mastery of Object-Oriented Programming project development activities.

Validation	Fails to demonstrate any understanding or implementation of validation forms or data validation techniques. There is no evidence of addressing validation requirements or dealing with data validation in their system.	Demonstrates limited understanding of validation forms and data validation techniques. While some attempt may be made to implement validation, it is incomplete or poorly executed, leading to inadequate handling of data validation.	Shows a basic understanding of validation forms and data validation techniques. They attempt to implement validation, but some aspects may be missing or incorrectly implemented, resulting in partial or inconsistent handling of data validation.	Demonstrates the use of validation forms and implements data validation techniques. Validation is mostly accurate and comprehensive, ensuring the proper handling of data input and verification in the system, although there may be minor issues.	Exhibits an exceptional understanding and implementation of validation forms and data validation techniques. Validation is meticulously implemented with thorough attention to detail, ensuring robust data validation procedures and contributing to the overall reliability and integrity of the system.
Verification	Fails to demonstrate any understanding or attempt to verify system data or functional requirements. There is no evidence of addressing verification requirements or considering data flow in their system solution.	Demonstrates limited understanding of verification processes and data flow in the system. Verification attempts may be incomplete or inaccurately applied, and there is insufficient consideration given to ensuring proper functional requirements and data flow.	Shows a basic understanding of verification processes and attempts to verify system data. However, verification efforts may lack thoroughness or precision, and there may be gaps in ensuring proper functional requirements and data flow within the system.	Effectively identifies and verifies system data, ensuring proper functional requirements are met. Verification efforts are mostly accurate and comprehensive, with attention to ensuring data integrity and appropriate data flow within the system.	Exhibits an exceptional understanding of verification processes and meticulously verifies system data. Verification efforts are comprehensive and precise, with a keen focus on ensuring all functional requirements are met and maintaining proper data flow throughout the system.

Table of Contents

- 1. INTRODUCTION
- 2. FEATURE LIST
 - 2.1 User 1: User
 - 2.2 User 2: Manager
 - 2.3 User 3: Admin
- 3. ER DIAGRAM
 - 3.1 Normalization
 - 3.2 Finalization
- 4. UML DIAGRAM
 - 4.1 Use Case Diagram
 - 4.2 Activity Diagram.
- 5. CONCLUSION

1. INTRODUCTION

The Smart Daily Task & Habit Management System (SDTHMS) is a desktop application developed using C# Windows Forms with Microsoft SQL Server. The system provides a single interface for managing daily tasks, habits, goals, and basic health metrics. Users can register, log in, view their dashboard, and maintain their personal records.

The application also contains Manager and Admin roles. A Manager logs in through the same Login form and is directed to the Manager Dashboard, where the manager can view system-level information, search users, view user-related activity data and send suggestions to a selected User. An Admin is authenticated using the configured Admin credentials and can manage User accounts through the Admin Dashboard.

The project applies object-oriented programming and GUI-with-database concepts covered in CSC2210. The implementation uses model classes, properties and methods, ADO.NET SQL Connection/SQL Command for SQL Server operations, parameterized database commands, BCrypt password hashing, and input validation.

The database design contains six related tables: Users, Tasks, Habits, Goals, Health Metrics and Manager Suggestions. The system uses primary and foreign keys to maintain relationships between user accounts and their related records.

2. FEATURE LIST

2.1 User 1: User

- Register an account with Username, Password, Mobile number and Address, with an optional profile image.
- Log in using the common Login form; the stored role determines the destination dashboard.
- View a personal Dashboard containing task, habit, goal, health and productivity information.
- Add, view and delete Tasks. Task categories are Work, Personal, Study, and General.
- Mark a Task as completed directly from the Dashboard.
- Add, view and delete Habits using Daily, Weekly or Monthly frequency.
- Increase a Habit streak from the Dashboard and update its last-complete date.
- Add, view and delete Goals and change their status between In Progress and Completed from the Dashboard.
- Add, view and delete Health Metrics containing water intake, sleep hours and time basis.
- View the latest Manager suggestions on the Dashboard and log out.

2.2 User 2: Manager

- Log in through the common Login form and access the Manager Dashboard.
- View system overview values for active Users, tracked Habits and task-completion rate.
- Switch between Users, Habits and Stats views.

- Search registered Users using the search field.
- Select a User from the displayed records and send a text suggestion.
- The suggestion is stored in ManagerSuggestions and can then be viewed by the target User.
- Log out and return to the Login form.

2.3 User 3: Admin

- Authenticate using the configured Admin username and password.
- View User accounts stored in the Users table.
- Add a User with Username, Password, Mobile, Address, Role and optional image.
- Update an existing User account.
- Delete a selected User account after confirmation.
- Clear the Admin input fields and log out.

2.4 Functional Requirements

ID	Requirement	Description
FR-01	User registration	The system shall validate required registration data and create a User account.
FR-02	Authentication	The system shall authenticate Admin, Manager, and User for access through the Login process.
FR-03	Task management	A User shall be able to add, view, delete and complete Tasks.
FR-04	Habit management	A User shall be able to add, view, delete and track Habit streaks.
FR-05	Goal management	A User shall be able to add, view, delete, and update Goal status.
FR-06	Health records	A User shall be able to add, view and delete Health Metrics.
FR-07	Manager supervision	A Manager shall be able to view system information, search Users, and send suggestions.
FR-08	Admin management	An Admin shall be able to add, view, update and delete User accounts.
FR-09	Database persistence	Application records shall be stored and retrieved using SQL Server.
FR-10	Input validation	The application shall reject required, invalid, or out-of-range input before database operations.

3. ER DIAGRAM

SDTHMS database contains six related tables. Users are the central entity. Tasks, Habits, Goals and HealthMetrics each reference Users (Id) through UserId. ManagerSuggestions references Users twice: UserId identifies the User receiving the suggestion and ManagerId identifies the Manager sending it.

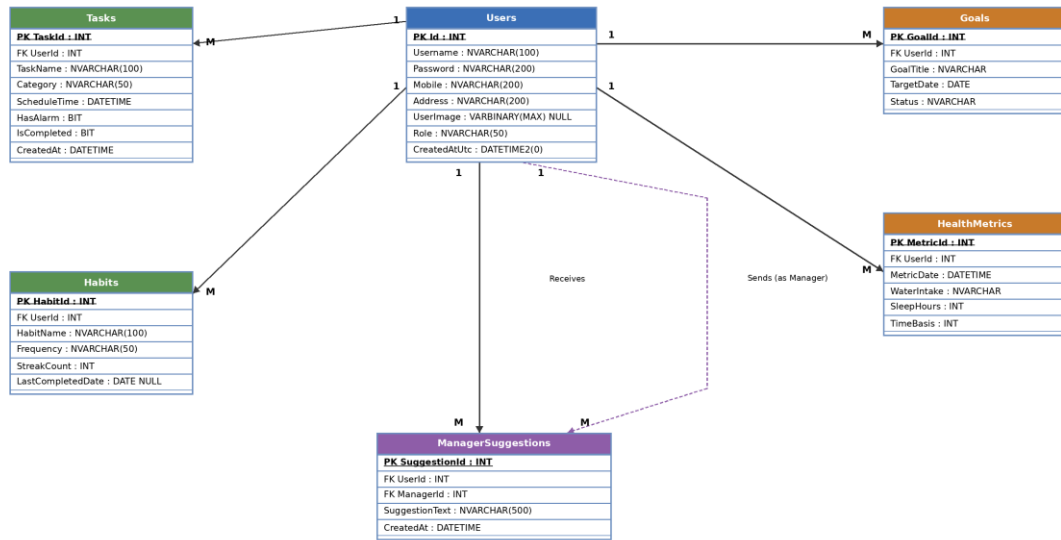


Figure 1: Entity Relationship Diagram of SDTHMS

3.1 Normalization

All six tables use a single column to surrogate primary key generated by SQL Server. Each field stores a single atomic value, and there are no repeating groups, satisfying First Normal Form (1NF).

Because every table has a single-column primary key, there can be no partial dependency on part of a composite key. Each non-key attribute depends on the complete primary key of its table, satisfying Second Normal Form (2NF).

The design also avoids storing attributes that depend on other non-key attributes within the same table. Therefore, the finalized structure is consistent with the project's normalized relational design.

3.2 Finalization

Table	Columns
Users	Id (PK), Username, Password, Mobile, Address, Role, UserImage, CreatedAtUtc
Tasks	TaskId (PK), UserId (FK → Users), TaskName, Category, ScheduleTime, HasAlarm, IsCompleted, CreatedAt
Habits	HabitId (PK), UserId (FK → Users), HabitName, Frequency, StreakCount, LastCompletedDate
Goals	GoalId (PK), UserId (FK → Users), GoalTitle, TargetDate, Status
HealthMetrics	MetricId (PK), UserId (FK → Users), MetricDate, WaterIntake, SleepHours, TimeBasis
ManagerSuggestions	SuggestionId (PK), UserId (FK → Users), ManagerId (FK → Users), SuggestionText, CreatedAt

4. UML DIAGRAM

4.1 Use Case Diagram

The Use Case Diagram identifies three actors: User, Manager and Admin. All three actors use the common Login process. User registration is available to the User. Users manage personal tasks, habits, goals and health records and view Manager suggestions. Managers view system information, search Users, inspect user-related activity information, and send suggestions. Admins manage User accounts.

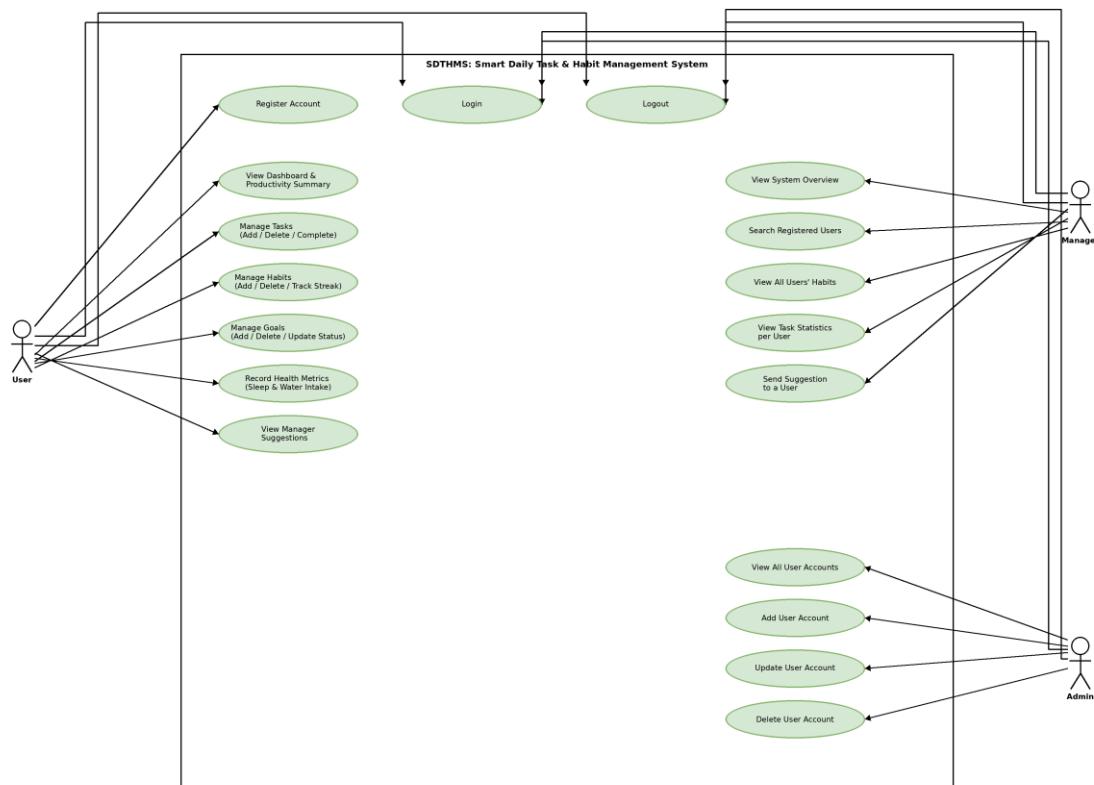


Figure 2: UML Use Case Diagram of SDTHMS

4.2 Activity Diagram.

The Activity Diagram represents the Login process implemented in Login.cs. The flow begins with username and password input, validates required fields, checks the configured Admin account, and otherwise retrieves the User record from SQL Server. BCrypt verification is then performed, and the stored Role determines whether the Manager Dashboard or User Dashboard is opened. Invalid credentials or invalid role values terminate the flow.

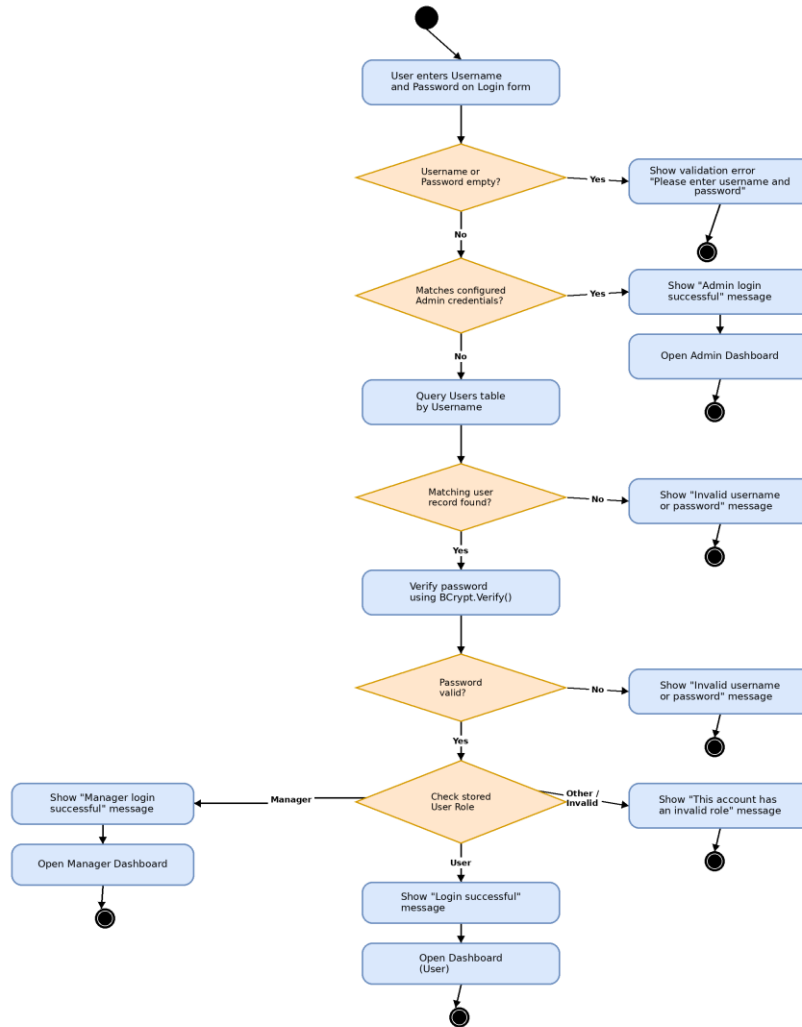


Figure 3: UML Activity Diagram — Login Process

5. CONCLUSION

The **Smart Daily Task & Habit Management System (SDTHMS)** was developed as a Windows Forms desktop application using **C# and Microsoft SQL Server** to provide an organized platform for managing daily tasks, habits, goals, health records, and productivity-related activities.

The system provides role-based access for **Users, Managers, and Administrators**. Users can manage their daily activities and health information, Managers can monitor user-related information and provide suggestions, and Administrators can manage user records. The

application integrates the graphical user interface with the SQL Server database to store and retrieve system data.

The project also applies important object-oriented programming concepts such as **classes, objects, properties, methods, encapsulation, inheritance/interfaces where applicable, exception handling, and database connectivity**. Input validation and parameterized SQL operations are used to improve data integrity and reduce invalid database operations.

Overall, SDTHMS fulfills the main objectives of the project by providing a practical desktop-based solution for daily task, habit, goal, and health management while demonstrating the required C# Windows Forms and database integration concepts.